4.1. Renaming Indexes and Columns

Previous Section:

3.3. Replacing & Mapping Values

Contents:

Before adding, removing, or updating data, it is often necessary to organize a DataFrame by giving meaningful names to its rows and columns. Clear column names improve code readability, while a well-chosen index makes data easier to locate, retrieve, and analyze.

By default, Pandas assigns integer indexes starting from **0**, but in many real-world datasets, a column such as a student ID, employee ID, or product code serves as a much more meaningful identifier. Likewise, descriptive column names make DataFrames easier to understand and maintain.

In this section, we continue using our synthetic student dataset to learn how to rename columns, customize indexes, and restore the default index when needed.

```
import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head(10))
```

1. Renaming Columns

Every DataFrame stores its column names in the `columns` attribute.

- `df.columns` returns an **Index** object containing all column names.
- `df.columns.to_list()` converts the Index object into a regular Python list.

```
print(df.columns)
print(df.columns.to_list())
```

Output:
Index(['Name', 'Age', 'Major', 'GPA'], dtype='object')
['Name', 'Age', 'Major', 'GPA']

1.1. Naming Columns When Creating a DataFrame

The `columns=` parameter of `pd.DataFrame()` specifies the column names.

Syntax: `pd.DataFrame(data, columns=[...])`

It has two possible behaviors:

- If a specified column **exists** in the input data, its values are included.
- If a specified column **does not exist**, Pandas creates that column and fills it with **NaN** values.

If `columns=` is omitted, Pandas automatically uses the keys of the input data as the column names.

1.2. Renaming Existing Columns

- The recommended way to rename columns is with `df.rename()`.

```
df.rename(columns={"GPA": "Grade"})
```

- Rename multiple columns simultaneously.

```
df.rename(columns={
    "Name": "Student",
    "Major": "Program",
    "GPA": "Grade"
})
```

- You can also replace all column names directly.

```
df.columns = ["Student", "Age", "Program", "Grade"]
```

This approach is convenient when changing every column name at once.

> 💡 **Note:** `df.rename()` returns a new DataFrame by default.
> To modify the original DataFrame, either assign the result back to `df` or use `inplace=True`. *The same idea can be applied when we impute missing values, insertion and deletion, or any modification made in the DataFrame*

2. Setting and Resetting the Index

Every DataFrame also has an **index**, which uniquely labels each row.

The index can be viewed using `print(df.index)` → Output is `RangeIndex(start=0, stop=20, step=1)`

Initially, Pandas assigns a default integer index from **0** to **N − 1**.

2.1. Why Set an Index?

A meaningful index makes data easier to organize and retrieve.

- Common choices include: Student ID; Employee ID; Product Code; Transaction Number

Using a unique identifier as the index also prevents it from being treated as a regular feature during preprocessing or machine learning.

- For example, encoding a unique ID column could produce hundreds or thousands of meaningless dummy variables that provide no predictive value.

2.2. Creating an Index During DataFrame Construction

The easiest way to define an index is when the DataFrame is first created.

Syntax: `pd.DataFrame(data, index=index_labels)`

For example:

```
pd.DataFrame(data, index=["S001", "S002", "S003"])
```

2.3. Setting an Existing Column as the Index

A column can later be promoted to the index using `set_index()`.

```
df = df.set_index("Name")
print(df.head())
```

Notice how the **Name** column becomes the row index and is no longer displayed as a regular column.

2.4. Restoring the Default Index

To convert the index back into a normal column, use `reset_index()`.

```
df = df.reset_index()
```

The default integer index is recreated, while the previous index becomes a regular column again.

2.5. Viewing Data as a Dictionary

After setting an index, it is often useful to convert the DataFrame into a dictionary.

```python
df.set_index("Name").to_dict()
```

Sample output:

```
{
   'Age': {
      'James': 20,
      'Chris': 18,
      ...
   },
   'Major': {
      'James': 'AI',
      'Chris': 'SW',
      ...
   }
}
```

The resulting dictionary uses the column names as keys, while the index labels become the keys of the nested dictionaries.

This representation is useful when exporting data or performing dictionary-based lookups.

Summary

This section covers various ways to rename or modify indexes and columns in a DataFrame

- Every DataFrame has **column names** (`df.columns`) and **row indexes** (`df.index`).
- `df.columns` returns an **Index** object, while `df.columns.to_list()` converts it into a Python list.
- The `columns=` parameter of `pd.DataFrame()` specifies column names during DataFrame creation and can also create missing columns filled with **NaN**.
- `df.rename()` renames one or more columns without modifying the original DataFrame unless assigned back or used with `inplace=True`.
- Assigning directly to `df.columns` replaces **all** column names at once.
- By default, DataFrames use a **RangeIndex** from **0** to **N – 1**.
- A meaningful index (such as a unique identifier) improves data organization and avoids treating identifier columns as regular features.

- `pd.DataFrame(index=...)` creates a custom index during DataFrame construction.
- `df.set_index()` converts an existing column into the index, while `df.reset_index()` restores the default integer index.
- `df.to_dict()` can convert a DataFrame into a dictionary, where the index is commonly used as the keys of nested dictionaries.

To practice further, complete Lab 04 of the Data Analytics Hands-On Lab.

🧪 Lab 07: Data Analytics with Python

References

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rename.html

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.reset_index.html

https://www.geeksforgeeks.org/pandas/how-to-rename-columns-in-pandas-dataframe/

https://www.geeksforgeeks.org/python/reset-index-in-pandas-dataframe/

Next Section:

📊 4.2. Inserting, Deleting, and Updating Data Values